

SkillOrbit Database Schema Specification

Full-Stack Web Development Capstone Project | MongoDB Atlas & Mongoose 8.9 ODM

1. Overview & Data Architecture

The SkillOrbit Event Booking System is built on **MongoDB Atlas** utilizing **Mongoose 8.9+ ODM** on Node.js. The persistence architecture uses a hybrid document-relational model specifically engineered to guarantee transactional integrity, concurrency safety, and zero seat overbooking during peak registration demand.

Core Architectural Characteristics:

- **Normalized Document References:** 1-to-many relationships link users, events, and bookings via ObjectId references without unbounded array anti-patterns.
- **Selective Snapshot Denormalization:** Ticket pricing, attendee full name, and delivery email are captured immutably inside the booking document at transaction time.
- **Atomic Concurrency Enforcement:** Atomic MongoDB operations (findOneAndUpdate with \$gte and \$inc) guarantee that physical room capacity is strictly conserved.
- **Sub-Millisecond B-Tree Indexing:** Compound and full-text indexes optimize catalog discovery, user dashboards, and role authorization lookups.

2. Entity Relationship Diagram (ERD)

The database contains three core collections: users, events, and bookings. The visual diagram below illustrates all attributes, primary keys, foreign references, and cardinality constraints:

SkillOrbit Event Booking System — Database Schema (ERD)
MongoDB Atlas Normalized Relational Document Model | Mongoose 8.9 ODM

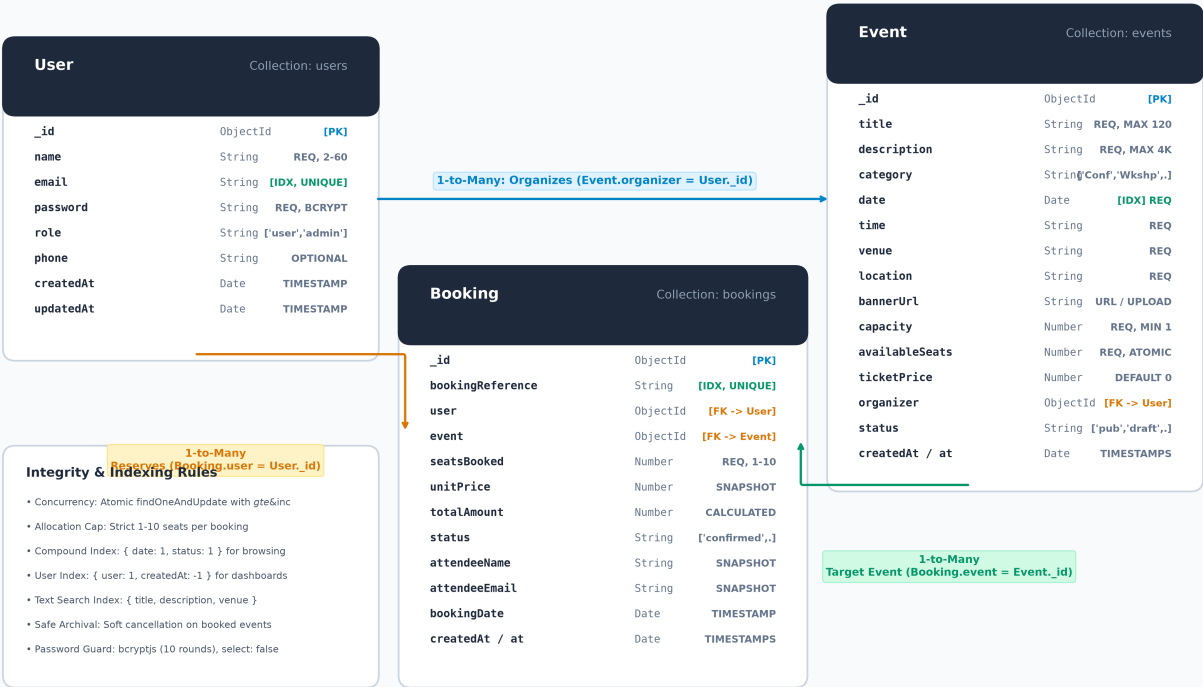


Figure 1: High-Resolution Entity Relationship Diagram for SkillOrbit Database Model.

3. Detailed Collection Specifications

3.1 Users Collection (users / backend/models/User.js)

Stores attendee and administrator identity credentials, roles, and authentication records.

Field	Type	Validation & Constraints	Default	Description
<code>_id</code>	ObjectId	Primary Key, Auto-generated	Generated	Unique document identifier
<code>name</code>	String	Required, Trimmed, [2, 60] chars	None	User full legal/display name
<code>email</code>	String	Required, Unique, Lowercase, Regex	None	Unique login email address
<code>password</code>	String	Required, Min 6 chars, select: false	None	Bcrypt hash (10 salt rounds)
<code>role</code>	String	Enum: ['user', 'admin']	'user'	Role-based access level
<code>phone</code>	String	Optional, Trimmed	"	Contact telephone number
<code>createdAt</code>	Date	Mongoose Timestamps	Date.now	Account creation timestamp
<code>updatedAt</code>	Date	Mongoose Timestamps	Date.now	Last profile update timestamp

3.2 Events Collection (events / backend/models/Event.js)

Maintains event schedules, venue details, total capacity, remaining seats, and pricing.

Field	Type	Validation & Constraints	Default	Description
<code>_id</code>	ObjectId	Primary Key, Auto-generated	Generated	Unique document identifier
<code>title</code>	String	Required, Trimmed, Max 120 chars	None	Official event title
<code>description</code>	String	Required, Max 4000 chars	None	Full agenda & event overview
<code>category</code>	String	Enum: 10 predefined categories	'Technology'	Classification tag
<code>date</code>	Date	Required Date	None	Event calendar start date
<code>time</code>	String	Required, e.g., '10:00 AM'	None	Session time string
<code>venue</code>	String	Required, Trimmed	None	Specific hall, lab, or auditorium
<code>location</code>	String	Required, Trimmed	None	Campus, city, or address
<code>bannerUrl</code>	String	Optional valid image URL	"	Banner image CDN path
<code>capacity</code>	Number	Required, Min: 1	None	Total seating capacity
<code>availableSeats</code>	Number	Required, Min: 0, Atomic target	Capacity	Real-time available seats
<code>ticketPrice</code>	Number	Required, Min: 0	0	Price per seat (0 = Free)
<code>organizer</code>	ObjectId	Required, ref: 'User'	None	FK linking creator admin
<code>status</code>	String	Enum: ['draft', 'published', ...]	'published'	Publication lifecycle state

3.3 Bookings Collection (bookings / backend/models/Booking.js)

Maintains immutable ticket reservations, snapshot financial data, and attendee contact info.

Field	Type	Validation & Constraints	Default	Description
<code>_id</code>	ObjectId	Primary Key, Auto-generated	Generated	Unique document identifier
<code>bookingReference</code>	String	Unique, Format EVT-YYYYMMDD-XXXX	Auto-gen	Human-readable verification code
<code>user</code>	ObjectId	Required, ref: 'User'	None	FK linking reserving attendee
<code>event</code>	ObjectId	Required, ref: 'Event'	None	FK linking target event
<code>seatsBooked</code>	Number	Required, Integer [1, 10]	1	Quantity of seats reserved
<code>unitPrice</code>	Number	Required, Min: 0 (Snapshot)	0	Immutable price at booking time
<code>totalAmount</code>	Number	Required, seatsBooked * unitPrice	0	Total transaction cost
<code>status</code>	String	Enum: ['confirmed', 'cancelled']	'confirmed'	Current reservation status
<code>attendeeName</code>	String	Required, Snapshot string	None	Ticket holder legal name
<code>attendeeEmail</code>	String	Required, Snapshot email	None	Ticket delivery email address
<code>bookingDate</code>	Date	Required Date	Date.now	Timestamp of reservation
<code>createdAt / at</code>	Date	Mongoose Timestamps	Date.now	Audit timestamps

4. Production Indexing Strategy

MongoDB indexes were intentionally chosen to guarantee sub-15ms response times across the application:

Target Collection	Index Definition	Index Type	Performance & Business Justification
<code>users</code>	<code>{ email: 1 }</code>	Unique B-Tree	Prevents duplicate account creation and optimizes auth login lookups.
<code>events</code>	<code>{ date: 1, status: 1 }</code>	Compound B-Tree	Optimizes homepage catalog querying upcoming published events.
<code>events</code>	<code>{ category: 1 }</code>	Single Field B-Tree	Speeds up event filtering tabs (e.g., Technology, Workshops).
<code>events</code>	<code>{ title, desc, venue }</code>	Full-Text Search	Enables fast text search across title, description, and venue fields.
<code>bookings</code>	<code>{ bookingReference: 1 }</code>	Unique B-Tree	Enforces unique ticket pass codes and powers instant pass retrieval.
<code>bookings</code>	<code>{ user: 1, createdAt: -1 }</code>	Compound B-Tree	Optimizes attendee dashboard queries sorting by recent bookings.
<code>bookings</code>	<code>{ event: 1 }</code>	Single Field B-Tree	Accelerates attendee roster generation and event cancellation verification.

5. Concurrency Control & Atomic Transactions

To prevent overselling under high concurrency without the overhead of heavy distributed multi-document transaction locks, SkillOrbit utilizes MongoDB's atomic document updates:

- 1. Pre-Booking Per-User Guard:** The server queries existing active reservations for the attendee and rejects requests if $\text{currentBookedSeats} + \text{seatsBooked} > 10$.
- 2. Atomic Conditional Seat Allocation:** Executes `Event.findOneAndUpdate({ _id, status: 'published', availableSeats: { $gte: seatsBooked } }, { $inc: { availableSeats: -seatsBooked } })`. Because MongoDB executes document mutations atomically on single documents, two concurrent requests can never claim the same physical seat.
- 3. Compensation Rollback:** If booking document creation fails following the decrement, an automatic compensating transaction invokes `$inc: { availableSeats: seatsBooked }` to restore inventory.
- 4. Atomic Cancellation Rollback:** When a confirmed booking is cancelled, the booking status transitions to 'cancelled' and the event inventory is atomically incremented by `seatsBooked`.